



Closures

Task

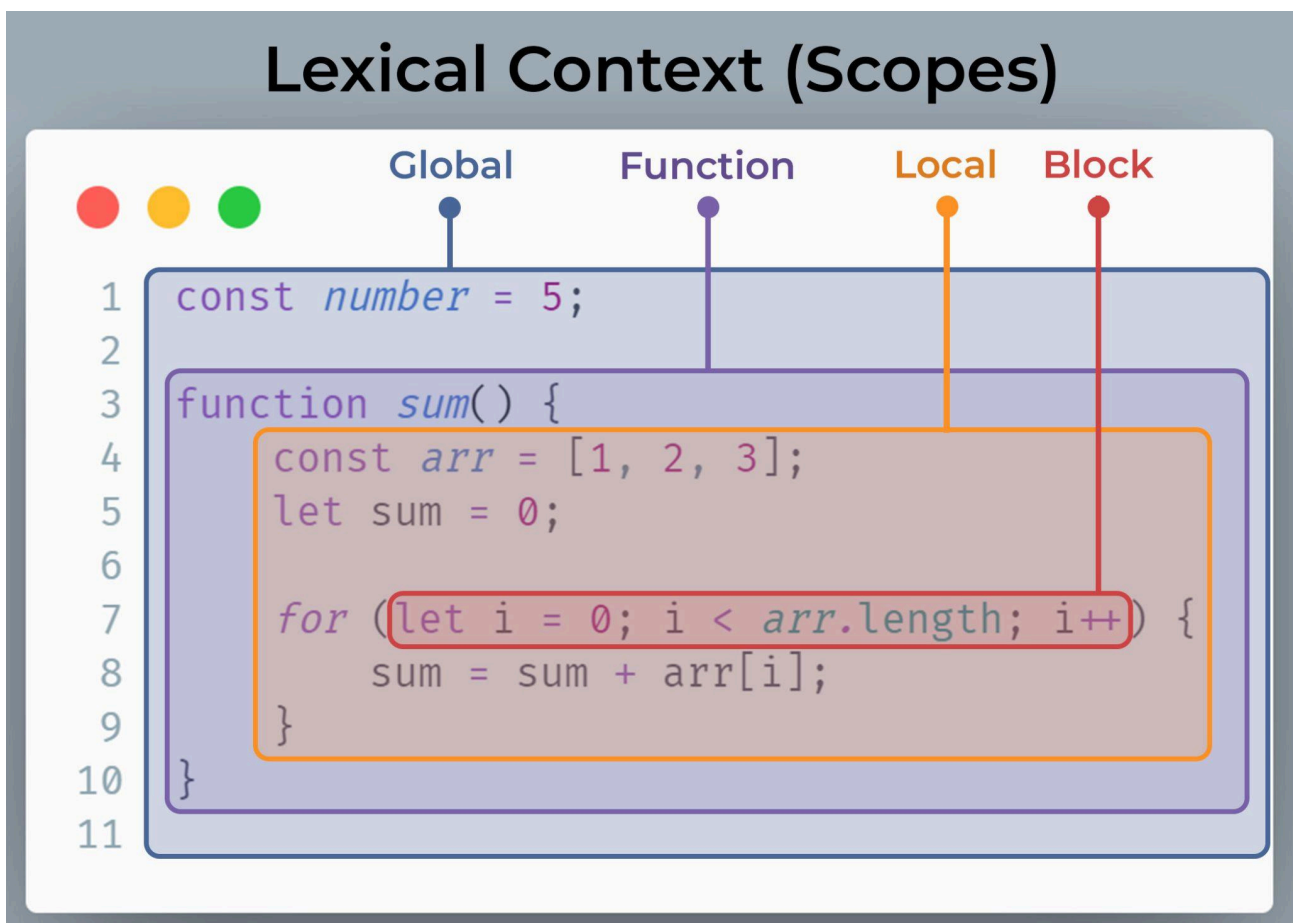
[Visit our website](#)

Introduction

Closures in JavaScript occur when an inner function has access to an outer function's variables and parameters, even after the outer function has finished executing. This is due to JavaScript's lexical scope, where a function retains access to its scope even after the function that created it has finished.

Lexical context

Lexical context (or lexical environment) refers to the surrounding context where code is physically written in the source code, and this directly determines how variables can be accessed. The image below demonstrates this through its nested boxes/scopes:



1. Global Context:

- Contains `const number = 5`, which is accessible to all inner contexts because of lexical nesting.
- Inner scopes (e.g., functions or loops) can 'see' and use `number`.

- Variables declared inside the function or loop cannot be accessed from the global scope.

2. **Function Context (function `sum()`):**

- Contains `const arr` and `let sum`, which are declared within the function and are scoped to it.
- Can access its own variables (`arr` and `sum`) and the global variable `number`.
- Creates its own lexical environment, meaning variables declared inside the function are scoped to the function.
- Variables declared here are not accessible to the global scope. For example, `arr` and `sum` cannot be accessed outside the function.

3. **Local Scope:**

- Refers to variables declared within a specific block (e.g., `if`, `for`, or `{}` inside a function).
- Variables declared with `let` or `const` in a block are limited to that block and cannot be accessed outside it.
- Difference from Function Scope:
 - Local scope applies to a smaller, block-specific context inside a function.
 - Function scope encompasses all variables declared in the entire function, even those not confined to a particular block.

4. **Block Context (for loop):**

- The `for` loop creates a block context for `let i`, meaning `i` is accessible only inside the loop.
- Can access **all** outer variables due to lexical nesting:
 - Can access `arr` and `sum` from function scope
 - Can access `number` from global scope
 - Has its own `i` variable that's only accessible within the loop
- Variables declared inside the block (e.g., `i`) are not accessible outside of it.

The key principle shown in the image above is that inner scopes have access to outer scopes based on their physical location in the code (lexical nesting), but not vice versa. For example, the `for` loop, being the innermost block, can access all variables defined in its enclosing scopes, such as the function scope or the global scope. Similarly, the `sum`

function can access global variables like `number` because it is defined in the outermost scope.

However, the reverse is not true, as the global scope cannot access variables declared inside the function or the `for` loop. This 'one-way' access pattern, represented by the nested boxes in the image, illustrates the core idea of lexical scoping: the location where code is written determines the variables and functions it can access.

Lexical scope's true power becomes apparent when functions carry their scope chain with them, creating what developers call "closures" – arguably JavaScript's most elegant and essential feature.

Why closures matter

Closures allow functions to 'remember' their lexical environment, even when called outside of their original scope. Closures are widely used for:

- Data encapsulation (e.g., creating private variables).
- Maintaining state across function calls.
- Handling asynchronous operations like event listeners or callbacks.

Returning functions from functions

A `closure` occurs when the returned function retains access to the variables and environment of its enclosing function, even after the outer function has completed execution. This allows the returned function to 'remember' and use those variables, making closures particularly useful for building flexible and reusable code.

Here's a step-by-step explanation of the closure example. Read through the code carefully:

1. Defining the `multiplier` function:

```
// The outer function "multiplier" that takes one parameter "x".  
function multiplier(x) {  
  // The nested function "inside" which takes one parameter "y".  
  function inside(y) {  
    // Return the product of "x" and "y".  
    return x * y;  
  }  
}
```

```

}

// Return the "inside" function. It has access to "x" from "multiplier".
return inside;
}

// Example usage:
const multiplyByThree = multiplier(3); // Creating a closure.
console.log(multiplyByThree(5)); // Outputs 15 which is 3 * 5.

```

- The `multiplier` function takes one parameter `x`.
- Inside `multiplier`, there is another function called `inside` that takes on the parameter `y`.
- The `inside` function returns the product of `x` and `y`.
- The `multiplier` function returns the `inside` function.

2. Creating a closure:

```

const multiplyByThree = multiplier(3); // Creating a closure.

```

- We call `multiplier(3)`, passing 3 as the argument for `x`.
- The `multiplier` function returns the `inside` function, with `x` set to 3.
- The returned `inside` function is stored in the constant `multiplyByThree`.
- `multiplyByThree` is a closure because it retains access to `x` from its lexical scope.

3. Using the closure:

```

console.log(multiplyByThree(5)); // Outputs 15 which is 3 * 5.

```

- We call `multiplyByThree(5)`, which is the `inside` function with `x` set to 3.
- Inside `multiplyByThree`, `y` is 5 (the argument passed).
- The function calculates `x * y`, which is `3 * 5`.
- The result, 15, is returned and logged to the console.

Summary:

- `multiplier(3)`: Returns a function (inside) with access to `x = 3`.
- `multiplyByThree(5)`: Uses `x = 3` and `y = 5` to compute `3 * 5`, resulting in 15.

The instance below is the same as the original example, with the addition of a step that outputs the `multiplyByThree` constant. This is to help you see what is meant when we say that the `inside` function is returned and assigned to the constant `multiplyByThree`. The code is also extensively commented on to help you identify exactly which parts are doing what.

```
// The outer function "multiplier" that takes one parameter "x".
function multiplier(x) {
  // The nested function "inside" which takes one parameter "y".
  function inside(y) {
    // Return the product of "x" and "y".
    return x * y;
  }
  // Return the "inside" function. It has access to "x" from "multiplier".
  return inside;
}

// Example usage:
// Create a closure with x = 3
const multiplyByThree = multiplier(3);

/* Logs the function itself. multiplyByThree is the inside function with x
set to 3. */
console.log(multiplyByThree)

/* Calls the inside function with y = 5. Since x was set to 3, it calculates
3 * 5 and logs 15. */
console.log(multiplyByThree(5));
```

Think of a closure as a backpack that the inner function carries with it. This backpack holds all the variables (like `x`) that the function had access to when it was created.

In the example, `multiplier` takes an argument `x` and returns a new function, `inside`, which multiplies `x` by its argument `y`. You can then use `multiplier` to create functions like `multiplyByThree`, which multiplies its input by 3.

The part of the code that calls `multiplier` and then `inside`:

```
// Think of it like: give me a function that multiplies three with whatever you give it  
  
const multiplyByThree = multiplier(3);  
  
console.log(multiplyByThree(5)); // Output: 15 (3 * 5)
```

- When `multiplier(3)` is called, the `multiplier` function executes and returns the `inside` function.
- The returned `inside` function still retains access to the parameter `x` from the `multiplier` function. In this case, `x` is 3.
- When the returned function is called with `multiplyByThree(5)`, the argument 5 is passed to the `inside` function as `y`.
- Within the `inside` function, the expression `x * y` would become `3 * 5`, which evaluates to 15.

The above instance could also be replaced with:

```
// We can also achieve the same result as following  
  
console.log(multiplier(3)(5)); // 15
```

- This single line works by directly calling the `multiplier` function with 3, which returns the `inside` function.
- Immediately after, it calls the returned `inside` function with 5 as the argument.
- The `inside` function, then calculates the product of `3 * 5`, which is 15.

Closures can be tricky at first, so if you're finding them confusing, that's perfectly okay! Keep practising and revisiting the concept, and it will become clearer over time.

Practical uses of closures

Closures are widely used in JavaScript programming to achieve functionality that would otherwise require more complex structures. Below are some practical use cases, explained in detail:

1. Creating private variables

Closures allow us to encapsulate data and create private variables, simulating the concept of data hiding. This is particularly useful when we want to prevent external code from directly accessing or modifying a variable:

```
function Counter() {  
  let count = 0; // Private variable.  
  return {  
    // Method to increment the count  
    increment: function () {  
      count++; // Increase the private 'count' variable  
      return count; // Return the updated count  
    },  
    // Method to decrement the count  
    decrement: function () {  
      count--; // Decrease the private 'count' variable  
      return count; // Return the updated count  
    },  
    // Method to retrieve the current count  
    getCount: function () {  
      return count; // Return the private 'count' variable  
    }  
  };  
}
```



```
// Create a new counter instance
const counter = Counter();
console.log(counter.increment()); // Output: 1 (count is increased by 1)
console.log(counter.increment()); // Output: 2 (count is increased by 1 again)
console.log(counter.decrement()); // Output: 1 (count is decreased by 1)
console.log(counter.getCount()); // Output: 1 (current count remains 1)
// Attempting to access 'count' directly will fail because it's private
// console.log(counter.count); // Output: undefined
```

The variable `count` is scoped to the `Counter` function and is not directly accessible outside of it. The inner functions (`increment`, `decrement`, `getCount`) form closures, allowing them to access and modify `count` safely.

2. Factory functions

A factory function is a design pattern where a function (the "factory") creates and returns other functions or objects, tailored with specific properties or behaviours. Closures are essential here because the returned function retains access to the parameters of the factory, enabling personalised or reusable functionality.

Let's create a factory function that generates personalised greeting functions for different occasions, like birthdays or holidays:

```
function greetingGenerator(occasion) {
  // Outer function takes an occasion and returns a new inner function
  return function(name) {
    // The inner function can use both 'occasion' and its own 'name' parameter
    return `Happy ${occasion}, ${name}!`;
  };
}

// Create specific greeting functions for different occasions
```

```
const birthdayGreeting = greetingGenerator("Birthday");
const holidayGreeting = greetingGenerator("Holidays");
// Use the generated functions
console.log(birthdayGreeting("Alice")); // Output: Happy Birthday, Alice!
console.log(holidayGreeting("Bob"));    // Output: Happy Holidays, Bob!
```

In the instance above:

- The `greetingGenerator` acts as a factory by creating new greeting functions, each customised for a particular occasion.
- Thanks to closures, each returned function retains access to its unique `occasion` value, allowing it to generate context-specific greetings.

This is a clean, modular way to build reusable functions for dynamic content creation.

3. Managing state across function calls

Closures are particularly useful for managing state in situations where a variable needs to persist across multiple function calls. For instance, a counter function can use a closure to ‘remember’ its current count even after the outer function has finished executing:

```
function createCounter() {
  let count = 0; // Private variable to keep track of the count
  // Return an inner function that modifies and returns the count
  return function() {
    count++; // Increment the count each time the function is called
    return count; // Return the updated count
  };
}

// Create a new counter instance
const counter = createCounter();
console.log(counter()); // Output: 1 (count starts at 0 and is incremented by 1)
```

```

console.log(counter()); // Output: 2 (count is incremented again)
console.log(counter()); // Output: 3 (count continues to increment)
// Each instance of createCounter has its own 'count' variable
const anotherCounter = createCounter();
console.log(anotherCounter()); // Output: 1 (this is a new counter instance)

```

In the above example, the `count` variable is private and only accessible through the returned function, ensuring encapsulation. Each call to the `createCounter` function creates a new closure with its own independent `count` variable, making it easy to keep state separate between instances.

4. Using closures for timed operations

Closures are invaluable for handling state in asynchronous operations, where variables might otherwise be lost or overwritten due to the non-linear execution of code. In this instance, we use closures to build a notification system that remembers which message to display next and ensures the correct timing for each log:

```

function notificationSystem(messages) {
  // Inner function to handle notifications, using recursion
  function notify(index) {
    // Check if there are more messages to display
    if (index < messages.length) {
      // Use setTimeout to schedule the notification display
      setTimeout(() => {
        console.log(messages[index]); // Log the current message
        notify(index + 1); // Call notify recursively for the next message
      }, 1000 * (index + 1)); // Delay increases with each message
    } else {
      // All messages have been sent
      console.log("All notifications sent!");
    }
  }
}

```

```

}

// Start the notification process with the first message
notify(0);
}

// Example usage
notificationSystem([
  "First notification", // This message will be shown after 1 second
  "Second notification", // This message will be shown after 2 seconds
  "Third notification" // This message will be shown after 3 seconds
]);

// Output:
// After 1 second: First notification
// After 2 seconds: Second notification
// After 3 seconds: Third notification

```

- The `notificationSystem` function creates a closure over the `messages` array.
- The `notify` function recursively schedules the display of each message with increasing delay.
- Thanks to closures, the `notify` function retains access to the `messages` array and the `index` variable between calls.

While closures showcase the power of JavaScript's lexical scoping, another feature that enhances the readability and conciseness of your code is **arrow functions**. Let's explore how they simplify function expressions and bring a modern touch to JavaScript programming.

Quick introduction to arrow functions

Arrow functions, a relatively recent addition to JavaScript, are a new way to write compact and clear functions that can simplify your code. First introduced with ES6, they provide a more concise syntax and a change in how **the keyword "this"** is handled in the function scope. You can learn more about arrow functions [here](#).

What are arrow functions in JavaScript?

Arrow functions in JavaScript are a shorthand syntax for writing function expressions. They're called “arrow” functions because of the `=>` symbol used, which resembles an arrow. They can make our code cleaner and easier to read when used correctly.

Syntax of arrow functions

The basic syntax of an arrow function is as follows:

```
let functionName = (parameter1, parameter2, ...parameterN) => expression
```

This creates a function named `functionName` that accepts parameters `parameter1`, `parameter2`, etc., through to `parameterN`, and returns the result of the expression. You can read this in English as something like “let `functionName` take `parameter1`, `parameter2`, etc, and then calculate `expression` with those parameters and return the result”.

For instance, here's a very basic arrow function:

```
let add = (a, b) => a + b;  
  
console.log(add(1, 2)); // Output: 3
```

This could be articulated in English as “let the `add` function take parameters `a` and `b`, and then add their values together and return the result”. The `let` keyword is not strictly necessary – try copying and pasting the code in the above instance and running it both with and without `let`, and see what happens.

An arrow function doesn't have to take any parameters. In such a case, you retain the parentheses, but without anything inside them, as shown below. Notice that when you output the `greeting` function without parentheses, the result depends on the environment. In most browsers, you typically see the code of the arrow function itself, while in Node.js, it usually displays the function type along with its name, such as `[Function: greeting]`. To actually execute the function and see its return value, you need to include parentheses when calling it, like this: `greeting()`.

```
let greeting = () => "Hello, World!";  
  
console.log(greeting); // Output: [Function: greeting] or () => "Hello, World!"
```

```
console.log(greeting()); // Output: Hello, World!
```

If you only have one parameter, you can even skip the parentheses! Here's an example of a one-parameter arrow function *with* parentheses:

```
let greeting = (name) => "Hello, " + name;  
console.log(greeting("Bob")); // Output: Hello, Bob
```

And here's the same arrow function *without* the parentheses:

```
let greeting = name => "Hello, " + name;  
console.log(greeting("Bob")); // Output: Hello, Bob
```

Limitations of arrow functions

Arrow functions have some unique behaviour, particularly around the `this` keyword, which can be confusing if not understood properly.

Here's why the section is useful:

1. No this Binding:

Arrow functions do not have their own `this`. Instead, they inherit `this` from the surrounding lexical context. This can cause issues when trying to use arrow functions as methods within objects, where `this` would not refer to the object itself.

```
const obj = {  
  value: 10,  
  method: () => console.log(this.value) // 'this' does not refer to the obj  
};  
obj.method(); // undefined
```

When we define the method using an arrow function (`method: () => console.log(this.value)`), it doesn't get its own `this` from the `obj` object.

Instead, the arrow function **inherits this from the surrounding lexical context**, which is the global context in this case (or the context where the `obj` was defined).

In the global context, `this.value` is `undefined`, because `value` is a property of `obj`, not the global object.

- If this were a regular function (i.e., `method: function() { console.log(this.value) }`), `this` would refer to `obj` when `obj.method()` is called, and the output would be `10`.

The behaviour of `this` in arrow functions can lead to confusion when you expect `this` to refer to the object (like `obj`), but instead, it refers to the **lexical context** in which the function was created, which in this case is the global context.

This is why arrow functions are not suited to be used as methods in objects when you need `this` to refer to the object itself.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Practical task

Follow these steps:

1. **Create a new JavaScript file** called `hide_and_seek.js`.
2. **Design a "Hiding Spot" function**
 - Create a function called `hide_and_seek`.
 - Inside `hide_and_seek`, define a variable `hiding_spot` that holds a secret hiding spot (e.g., "under the bed", "in the closet", etc.).
 - Inside `hide_and_seek`, define another function called `guess_location` that takes a guess as input and checks if the guess matches the `hiding_spot`.
 - Return `guess_location` from `hide_and_seek`.
3. **Play the game**
 - Call `hide_and_seek` to get the `guess_location` function.
 - Use a loop to let the user guess multiple times.

- After each guess, tell the user if they found the hiding spot or not.

4. Celebrate success

- When the user guesses correctly, congratulate them and break out of the loop.

Example output:

```
Welcome to Hide and Seek!
Where do you think I'm hiding?
Guess: in the closet
Nope, that's not it.
Guess: under the bed
You found me! Great job!
```

Hints:

- Use a closure to 'hide' the `hiding_spot` inside `hide_and_seek`.
- Ensure `guess_location` has access to `hiding_spot` without exposing it outside.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback [form](#) to help us ensure we provide you with the best possible learning experience.
